

Towards High-DoF Dexterous Manipulation through VLA Post-Training

Journal Title
XX(X):1–27
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/


Junlei Zhu^{1,2*}, Shenzhe Yao^{1*}, Chaogui Huang¹, Wenkai Zhu^{1,2},
Jingwei Peng^{1,2}, Guanqi He¹, Sören Schwertfeger², Jiahao Chen¹ and
Yide Liu¹

Abstract

Imitation-learned vision–language–action (VLA) foundation models acquire broad manipulation capabilities by scaling robot data across tasks and embodiments, but reliable deployment on a specific downstream task and hardware platform still requires post-training. Dexterous hands make this adaptation particularly difficult: their broad behavioural repertoire and high degree of freedom create a large and structured action space. Three obstacles are central: open-source VLAs do not natively provide an action interface for high-DoF hands; gesture mismatch during human-gated DAgger takeover creates command discontinuities and contaminates corrective trajectories; and reinforcement learning in the raw joint space is sample-inefficient. We present a unified four-step post-training pipeline comprising a learned temporal hand-action codec, supervised fine-tuning, DAgger, and real-world residual reinforcement learning. The codec adapts a pretrained VLA to absolute dexterous-hand commands. Buffered rollback, pose alignment, and smooth command blending enable continuous, task-relevant DAgger corrections, while latent residual RL confines exploration to coordinated hand motions captured by the codec. We evaluate the pipeline on five diverse real-world tasks spanning bimanual transfer, in-hand reorientation, and tool use. Within the reported post-training budgets, the resulting policies achieve 100% success on every evaluated task over 20 trials per task. These results provide a practical path for adapting VLA foundation models to reliable real-world dexterous manipulation.

Keywords

Dexterous manipulation, real-world reinforcement learning

1 Introduction

Vision–language–action (VLA) foundation policies are increasingly capable across tasks and embodiments (Brohan et al. 2023; Open X-Embodiment Collaboration 2024; Octo Model Team et al. 2024; Black et al. 2024; Physical Intelligence et al. 2025). Despite this progress, deployment on a particular downstream task and hardware platform can

¹Wuji Technology

²School of Information Science and Technology, ShanghaiTech University, China

*These authors contributed equally to this work.

Corresponding author:

Yide Liu, Wuji Technology.

Email: liu.yide@wuji.tech

Project website: wuji.tech/blog/post-training-0.

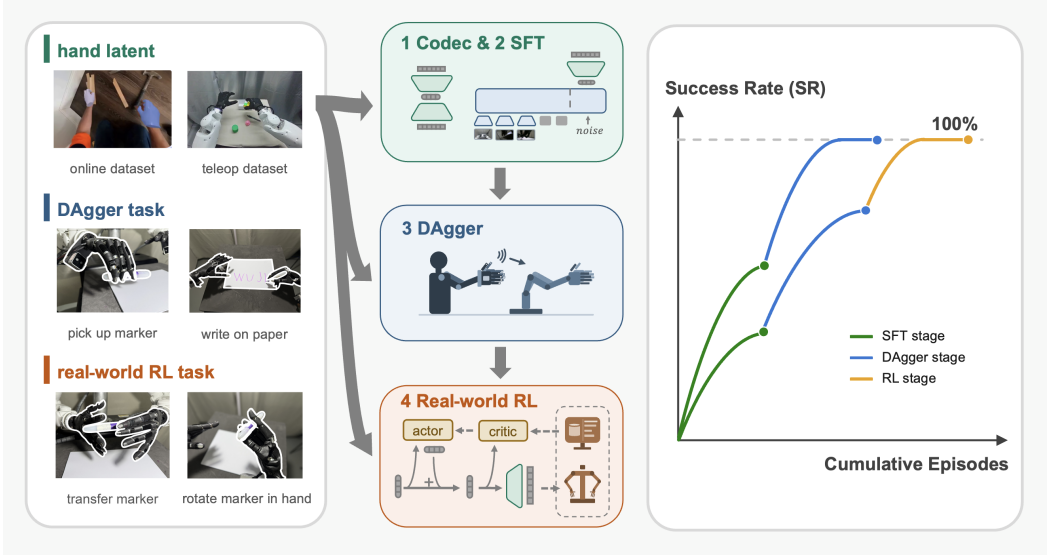


Figure 1. Overview of dexterous policy post-training. A learned hand-action representation provides a common interface for supervised fine-tuning, corrective data aggregation, and real-world latent residual reinforcement learning, which progressively improve task success. The curves on the right schematically illustrate improvement across post-training stages.

still expose systematic failures caused by differences in task distribution, embodiment, sensing, action interfaces, and encountered states. Task- and robot-specific post-training is therefore needed to turn these broad capabilities into reliable execution.

Dexterous manipulation makes this adaptation particularly challenging. Compared with parallel grippers, dexterous hands support a broader range of manipulation behaviors, expose many more controlled degrees of freedom, and require coordinated finger postures under multi-contact dynamics (Rajeswaran et al. 2018; OpenAI et al. 2020). The resulting action space is both high-dimensional and strongly structured. This creates three obstacles across the post-training pipeline: establishing a compatible action interface, collecting clean human corrections, and exploring useful policy improvements efficiently on the real robot.

The first obstacle is the mismatch between the pretrained VLA’s action interface and

high-DoF hand control. The pretrained policy’s fixed action output does not directly match the joint-command space of our bimanual dexterous embodiment. We therefore train a temporal hand codec that maps coordinated hand trajectories into a compact latent action space compatible with the VLA.

The second obstacle is gesture mismatch during DAgger intervention, which causes discontinuous high-DoF hand commands and contaminates corrective supervision. Recent dexterous human-in-the-loop systems have identified intervention discontinuity in high-DoF control, and HandITL attributes it to command mismatch between the operator and the policy and avoids it through relative hand retargeting and shared arm control (Han et al. 2026; Li et al. 2026b). We study a different interface constraint: both teleoperation and the reference VLA use absolute hand commands, which we retain so that demonstrations, corrections, and policy outputs share the same action semantics. Under this interface, directly

Table 1. Important symbols used throughout the paper, grouped by pipeline stage.

Symbol	Brief description	Symbol	Brief description
<i>Step 1: Hand-action codec</i>			
H	Action-chunk horizon ($H = 32$)	$E^{\text{hand}}, D^{\text{hand}}$	Per-hand codec encoder and decoder
$q, q_0 \in \mathbb{R}^{54}$	Current state; chunk-start state	E, D	Full action-codec encoder and decoder
$q_0^{\text{hand}} \in \mathbb{R}^{20}$	Hand state at chunk start	$z_{1:H}^{\text{hand}} \in \mathbb{R}^{H \times 9}$	Per-hand latent chunk
$q_{1:H}^{\text{hand}, \text{cmd}} \in \mathbb{R}^{H \times 20}$	Commanded hand-joint chunk	$z_{1:H} \in \mathbb{R}^{H \times 32}$	Full latent-action chunk
$q_t^{\text{cmd}} \in \mathbb{R}^{54}$	Executable joint command	μ, σ, ϵ	VAE mean, std., and sampling noise
$a_{1:H} \in \mathbb{R}^{H \times 54}$	Joint-space action chunk	$\Delta^k, e_{t,j}^{(k)}$	Temporal-difference operator; error
$a_{1:H}^{\text{hand}} \in \mathbb{R}^{H \times 20}$	Relative hand-action chunk	$\tau_{\text{Huber}}, \beta_{\text{KL}}$	Huber threshold and KL weight
$\hat{a}_{1:H}^{\text{hand}} \in \mathbb{R}^{H \times 20}$	Reconstructed hand-action chunk	w_j, λ_k	Joint and temporal loss weights
<i>Step 2: Supervised fine-tuning</i>			
$C = 25$	Executed frames per policy query	$o = (I_{1:K}, \ell, q)$	Observation: views, language and state
$t_{\text{flow}} \in [0, 1]$	Flow time	$x_{t_{\text{flow}}} \in \mathbb{R}^{H \times 32}$	Interpolated latent
$\hat{z} \in \mathbb{R}^{H \times 32}$	One-step clean-latent estimate	v_ω, u	Predicted vector field and flow target
$\lambda_{\text{arm}}, \lambda_{\text{lat}}, \lambda_{\text{dec}}$	SFT loss weights		
<i>Step 3: DAgger</i>			
$q^{\text{hold}} \in \mathbb{R}^{20}$	Pre-takeover hand command	$q_t^{\text{teleop}} \in \mathbb{R}^{20}$	Live operator command
s	Elapsed blend time	τ_b	Blend duration
α_s	Blend fraction at elapsed time s	α_{new}	New-correction batch fraction
<i>Step 4: Real-world residual RL</i>			
π_{ref}	Frozen reference policy	$\mathcal{B}$	Replay buffer
$z_{\text{ref}} \in \mathbb{R}^{H \times 32}$	Reference-policy latent chunk	r_t, d_t, γ	Reward, terminal flag and discount
$\eta_{1:M} \in \mathbb{R}^{M \times 2048}$	Prefix image-token features	$Q_{\phi_i}, Q_{\bar{\phi}_i}$	Twin critics and targets
$\xi_{\text{rl}} \in \mathbb{R}^{2048}$	Learnable readout token	τ_{Polyak}	Target averaging coefficient
g_φ, W_{rl}	Readout transformer/projection	$\mathcal{R}_{\Delta z}, \mathcal{S}$	Residual/smoothness penalties
$h_{\text{rl}} \in \mathbb{R}^{2048}$	RL-token observation representation	$\beta_q, \beta_{\Delta z}, \beta_s$	Actor-loss weights
$f_\theta, f_{\bar{\theta}}$	Actor and TD3 target actor	b^c, b^z	Correction and latent bounds
$c_{1:C}, \tilde{c}_{1:C} \in \mathbb{R}^{C \times 32}$	Bounded and noise-perturbed corrections	$\sigma^{\text{arm}}, \sigma^{\text{hand}}$	Exploration noise scales
$\Delta z \in \mathbb{R}^{H \times 32}$	Full-horizon latent residual	ν	Noise spectral exponent ($\nu = 2$)
$m_{\text{task}} \in \{0, 1\}^{32}$	Binary residual mask	$\dot{q}_{\text{max}}$	Maximum command change per frame

switching to teleoperation when the operator’s live gesture differs from the robot hand’s current commanded pose produces a discontinuous joint command. Beyond the physical discontinuity, the initial recorded motion may describe gesture realignment rather than recovery from the policy failure, contaminating the corrective supervision. We therefore separate pose synchronization from task recovery by rolling the robot back to a buffered pre-failure state, displaying the operator and robot hand poses for alignment, and blending the hand command

over 2 s at the control switch. This mechanism preserves absolute hand control while producing continuous, task-relevant correction trajectories.

The third obstacle is sample-inefficient exploration in the high-dimensional raw joint space of a dexterous hand. As the number of independently controlled joints increases, online reinforcement learning must search over a rapidly expanding space of possible corrections, making effective behaviors difficult to discover from limited interaction (Rajeswaran

et al. 2018). This problem is compounded by contact-rich dynamics, for which simulation is difficult to model accurately and real-robot samples are expensive (OpenAI et al. 2020, 2019; Xu et al. 2026). Consequently, dexterous post-training calls for an off-policy method that can reuse collected experience while making each online correction structurally meaningful (Lillcrap et al. 2016; Fujimoto et al. 2018). We meet this requirement with latent residual reinforcement learning, which searches for corrections in a compact hand-action representation learned from coordinated motion rather than directly perturbing individual joint coordinates (Santello et al. 1998; He and Ciocarlie 2021).

Our goal is to turn a pretrained VLA into a reliable policy for a specified real-world dexterous skill. We focus on atomic tasks that can be completed in approximately 30 s; long-horizon task composition and scene-level generalization are outside the scope of this work. To this end, we introduce a four-step post-training pipeline. We first train and freeze a temporal codec that represents coordinated hand motion in a compact latent action space (Santello et al. 1998; Kingma and Welling 2014; He and Ciocarlie 2021). We then fine-tune the VLA with supervised learning in this space, aggregate human corrections through DAgger, and finally freeze the corrected policy and learn a latent residual through online reinforcement learning (Ross et al. 2011; Johannink et al. 2019; Xu et al. 2026). Together, these stages provide a progression from task initialization with demonstrations, through correction on policy-induced states, to reward-driven improvement from real-world interaction.

We evaluate the pipeline on five real-world marker-manipulation tasks spanning grasping, bimanual transfer, in-hand rotation, object separation, and writing. Starting from 200 demonstrations per task, we apply DAgger and latent residual RL only when the preceding stage leaves systematic failures. The successive stages raise every task to 100% success over 20 evaluation trials.

We make four contributions.

- First, we present an end-to-end post-training pipeline that integrates SFT, DAgger, and latent residual RL through a shared hand-action representation for real-world dexterous skills.
- Second, we formulate the DAgger hand-off problem under an absolute hand-command interface and identify two coupled consequences of gesture mismatch: executed-command discontinuity and contamination of corrective supervision by gesture-realignment motion.
- Third, we formulate latent residual RL that refines the actions of a frozen VLA through corrections in the learned hand-action space.
- Fourth, we conduct extensive real-robot experiments that evaluate the complete pipeline, quantify the gains obtained at successive post-training stages, compare DAgger corrections with additional demonstrations, and compare latent with raw joint-space residual learning under matched budgets.

Videos of the real-robot results are available on the [project website](#).

2 Related Work

2.1 Foundation policies and dexterous action representations

Large-scale robot datasets and vision-language-action (VLA) models have produced increasingly capable policies that transfer across tasks, embodiments, and semantic instructions (Brohan et al. 2023; Open X-Embodiment Collaboration 2024; Octo Model Team et al. 2024; Black et al. 2024; Physical Intelligence et al. 2025). Task- and robot-specific deployment nevertheless commonly begins with supervised fine-tuning on demonstrations. Dexterous manipulation makes this adaptation especially difficult because useful hand actions are high-dimensional but strongly coordinated. Biomechanical studies

identify low-dimensional postural synergies in human grasps (Santello et al. 1998), and robotic methods learn related action synergies across manipulation tasks (He and Ciocarlie 2021). Human-video pretraining pipelines provide a further source of structured hand-motion supervision for robot-policy training (Li et al. 2026a).

Earlier dexterous-learning systems combine demonstrations with reinforcement learning or use large-scale simulation and domain randomization to acquire task-specific hand skills (Rajeswaran et al. 2018; OpenAI et al. 2020, 2019). These results establish the feasibility of learned dexterity, but do not provide a shared post-training action representation for a pretrained VLA. Our temporal hand codec is trained once and then frozen, providing a common action interface for supervised adaptation, interactive correction, and real-world reinforcement learning.

2.2 Interactive correction for dexterous policies

Dagger addresses compounding imitation-learning error by collecting expert labels under the learner-induced state distribution (Ross et al. 2011), while human-gated variants concentrate supervision on states at which an operator elects to intervene (Kelly et al. 2019). For real-robot deployment, Sirius uses intervention events to reweight mixed human and robot experience during continual policy learning (Liu et al. 2025a). Recent dexterous VLA systems bring this paradigm to coordinated arm-hand control. DexHiL identifies intervention discontinuity as a challenge in high-DoF dexterous control and combines an integrated teleoperation interface with intervention-aware sampling (Han et al. 2026). HandITL further attributes gesture jumps at takeover to command mismatch between human teleoperation and policy execution, and avoids them through relative hand retargeting and shared arm control (Li et al. 2026b).

We retain the absolute hand-command interface shared by teleoperation and the reference

VLA. At takeover, a mismatch between the policy and operator commands can both produce a discontinuity in the executed command and introduce pose-synchronization motion into the correction data. We address these effects through buffered rollback, pre-takeover pose alignment, and a 2-s blend of the executed hand command, without changing the action semantics.

2.3 Frozen-policy reinforcement learning and latent action spaces

Residual reinforcement learning preserves an existing controller and learns a corrective action on top of it (Johannink et al. 2019). Recent VLA post-training systems use experience in several different ways. $\pi_{0.6}^*$ and its RECAP procedure combine demonstrations, autonomous rollouts, and interventions through offline RL and advantage-conditioned policy learning (Amin et al. 2025). RL Token instead freezes the VLA, reads out a compact backbone feature, and trains small online actor-critic networks (Xu et al. 2026). BORA combines an offline action-conditioned critic with online action-residual adaptation for real-world dexterous VLAs (Chen et al. 2026).

A complementary line of work asks where policy improvement should act. PLAS constrains offline RL to the support of a learned action decoder (Zhou et al. 2021), and LASER learns task-family action manifolds for more efficient simulated RL (Allshire et al. 2021). For frozen generative robot policies, DSRL steers the diffusion or flow noise latent (Wagenmaker et al. 2025), whereas ZPRL perturbs an observation-side information bottleneck that conditions the frozen action generator (Yu et al. 2026). Our residual acts instead in an action-side hand codec shared with the VLA’s supervised training. The reference policy remains frozen, a zero residual exactly recovers its decoded action, and the same real-robot budget can be used to compare latent and raw joint-space residuals.

3 Method

Our pipeline consists of four steps. We first train a codec that compresses dexterous hand motion into a low-dimensional latent, and validate that its decoder faithfully reconstructs feasible hand motion (Step 1). We then fine-tune a vision-language-action policy that acts in that latent space (Step 2). Next, we refine the policy through corrective data aggregation (Step 3). Finally, we freeze that policy and use online residual reinforcement learning (RL) to learn a correction confined to the latent action space (Step 4). Figure 2 summarizes the full pipeline. Table 1 summarizes the principal notation used below.

Step 1: Training the Codec

Action space. We work with a bimanual platform whose two 7-DoF arms carry two 20-DoF dexterous hands. We represent a single-frame joint-space action by $a \in \mathbb{R}^{54}$, ordered as

$$a = (a_L^{\text{arm}}, a_L^{\text{hand}}, a_R^{\text{arm}}, a_R^{\text{hand}}). \quad (1)$$

The four blocks have dimensions 7, 20, 7 and 20, respectively. Action chunks preserve this ordering at every time step, and the measured proprioceptive state $q \in \mathbb{R}^{54}$ follows the same ordering.

Chunk-level hand action encoding. Motivated by the low-dimensional postural synergies observed in human grasps (Santello et al. 1998), we compress each hand-action chunk from $\mathbb{R}^{H \times 20}$ to a latent sequence in $\mathbb{R}^{H \times 9}$ using a variational autoencoder (Kingma and Welling 2014) trained separately for each hand and frozen thereafter. We describe one hand below and suppress the side label. Let $q_0 \in \mathbb{R}^{54}$ be the measured joint configuration at the start of the chunk, and let $q_0^{\text{hand}} \in \mathbb{R}^{20}$ denote the corresponding hand block. Let $q_{1:H}^{\text{hand,cmd}} \in \mathbb{R}^{H \times 20}$ denote the commanded joint-position chunk for that hand. We represent it relative to

the initial hand configuration:

$$a_t^{\text{hand}} = q_t^{\text{hand,cmd}} - q_0^{\text{hand}}, \quad t = 1, \dots, H. \quad (2)$$

Each joint coordinate of $a_{1:H}^{\text{hand}}$ is standardized before encoding; we omit this preprocessing from the notation. Encoder and decoder are small temporal Transformers that attend across all H frames of a chunk; their architecture is summarized in Table 3 of Appendix A. They use temporal embeddings fixed to H , and we apply the codec only to full-horizon chunks. The encoder emits a per-frame Gaussian posterior, and the decoder reconstructs the hand action:

$$\begin{aligned} (\mu, \sigma) &= E^{\text{hand}}(a_{1:H}^{\text{hand}}, q_0^{\text{hand}}), \\ z_{1:H}^{\text{hand}} &= \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \\ \hat{a}_{1:H}^{\text{hand}} &= D^{\text{hand}}(z_{1:H}^{\text{hand}}, q_0^{\text{hand}}), \end{aligned} \quad (3)$$

with $\mu, \sigma, z_{1:H}^{\text{hand}} \in \mathbb{R}^{H \times 9}$. The codec is trained with

$$\begin{aligned} \mathcal{L}_{\text{codec}} &= \sum_{k=0}^2 \lambda_k \mathcal{L}_{\text{WH}}^{(k)} \\ &\quad + \beta_{\text{KL}} \text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I)), \end{aligned} \quad (4)$$

Here $\Delta^0 x_t = x_t$ and, for $k \in \{1, 2\}$, $\Delta^k x_t = \Delta^{k-1} x_{t+1} - \Delta^{k-1} x_t$; thus $\Delta^1 x_t = x_{t+1} - x_t$ and $\Delta^2 x_t = x_{t+2} - 2x_{t+1} + x_t$. All differences and errors are computed in the standardized relative-action space. For temporal order $k \in \{0, 1, 2\}$, define the error at each valid frame $t \in \{1, \dots, H - k\}$ and joint j as

$$e_{t,j}^{(k)} = (\Delta^k \hat{a}^{\text{hand}})_{t,j} - (\Delta^k a^{\text{hand}})_{t,j}. \quad (5)$$

The per-joint weighted Huber term is

$$\mathcal{L}_{\text{WH}}^{(k)} = \frac{1}{20(H-k)} \sum_{t=1}^{H-k} \sum_{j=1}^{20} w_j \ell_{\tau_{\text{Huber}}} \left(e_{t,j}^{(k)} \right), \quad (6)$$

where w_j is the weight of joint j and the scalar Huber penalty (Huber 1964) is

$$\ell_{\tau_{\text{Huber}}}(e) = \begin{cases} \frac{1}{2} e^2, & |e| \leq \tau_{\text{Huber}}, \\ \tau_{\text{Huber}} (|e| - \frac{1}{2} \tau_{\text{Huber}}), & |e| > \tau_{\text{Huber}}. \end{cases} \quad (7)$$

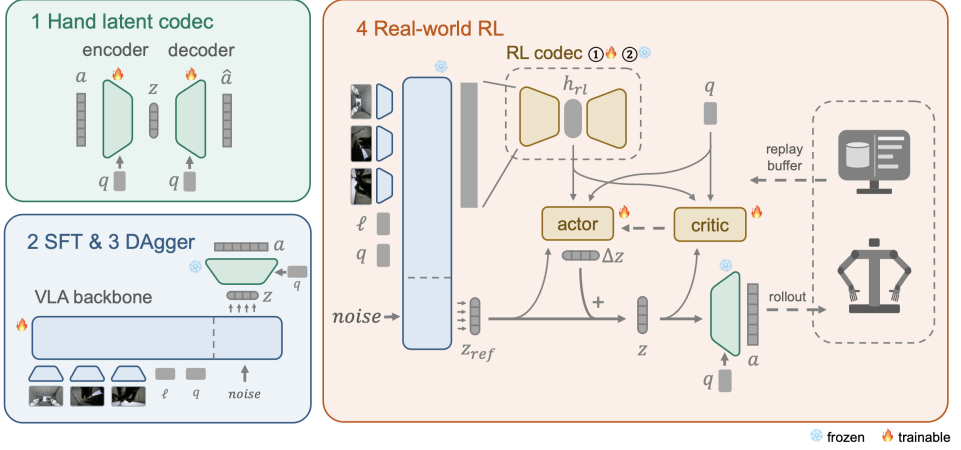


Figure 2. Overview of our pipeline. **Codec.** A chunk-level VAE is trained per hand on teleoperation, task and retargeted human-video data, compressing each hand from 20 to 9 dimensions; the resulting codec (E, D) is frozen. **SFT and DAgger.** Demonstrations are expressed through E as latent supervision targets, and the VLA is fine-tuned to emit latent actions in that space, with DAgger-style operator takeovers folded back into the corpus. The policy is frozen thereafter. Here, ℓ denotes the language instruction and q the measured proprioceptive state. **Real-world RL.** The RL-token encoder is trained on demonstration data before online RL ① and kept frozen during online RL ②. Online, the frozen policy emits z_{ref} with no encoding step, the RL token h_{r1} is read out from the frozen prefix, and an actor conditioned on $(h_{r1}, q, z_{\text{ref}})$ adds a bounded latent residual Δz that is decoded once into the executed chunk. Rollouts fill a replay buffer from which the critics and the actor are trained.

Its quadratic branch remains sensitive to typical reconstruction errors, while its linear branch prevents occasional large deviations in the heterogeneous training corpus from dominating the update. The $k = 0$ term preserves joint positions, and the $k = 1, 2$ terms additionally preserve frame-to-frame motion and changes in that motion. The threshold τ_{Huber} , joint weights w_j , and temporal weights λ_k are specified in Appendix B. The KL term regularizes the posterior toward $\mathcal{N}(0, I)$, fixing the latent scale and discouraging vanishing posterior variance; both are useful under the latent perturbations introduced in Step 4. At inference, we use the posterior mean, $z_{1:H}^{\text{hand}} = \mu$.

Combining the two hand codecs with the arm blocks gives the $54 \leftrightarrow 32$ codec (E, D). For a full action chunk $a_{1:H} \in \mathbb{R}^{H \times 54}$, the encoded chunk $z = E(a_{1:H}; q_0) \in \mathbb{R}^{H \times 32}$ has the block

structure

$$z = (z_L^{\text{arm}}, z_L^{\text{hand}}, z_R^{\text{arm}}, z_R^{\text{hand}}). \quad (8)$$

Each arm block is the corresponding $H \times 7$ action block, while each $H \times 9$ hand block is produced by equation 3; thus the codec constrains only the hand degrees of freedom.

The decoder produces the uncompressed joint-space action $a_{1:H} = D(z; q_0)$. Adding the starting state to each frame forms the executable command,

$$q_t^{\text{cmd}} = q_0 + a_t, \quad t = 1, \dots, H.$$

Thus a denotes the underlying joint-space action, whereas the learned policy operates in its encoded representation z .

Step 2: SFT

Policy in the latent action space. We fine-tune the open-source $\pi_{0.5}$ flow-matching vision-language-action model (Physical Intelligence et al. 2025). It maps an observation

$$o = (I_{1:K}, \ell, q), \quad (9)$$

to an action chunk of horizon $H = 32$, where $I_{1:K}$ are RGB images from $K = 3$ cameras, ℓ is the language instruction, and q is the proprioceptive state (Zhao et al. 2023). The observation is processed by a pretrained vision-language backbone, while a noisy action chunk is carried by a smaller *action expert* that attends to the backbone tokens and is trained by conditional flow matching (Lipman et al. 2023). A chunk is generated by integrating the predicted vector field from noise over a small number of Euler steps. Inference is synchronous: of each chunk only the first $C = 25$ frames are executed before the next observation is taken, which trades the temporal consistency of open-loop chunk execution against reactivity (Liu et al. 2025b; Zhao et al. 2026).

Rather than the 54-dimensional joint-space action, the policy predicts the 32-dimensional representation of Eq. 8, yielding $z = \pi(o) \in \mathbb{R}^{H \times 32}$. At deployment the decoder reconstructs $a = D(z; q)$, and the robot executes $q + a_t$ at each frame of the chunk. The encoder E is used only during training to convert joint-space demonstrations into latent supervision targets.

Supervising in both spaces. Because the hand decoder is nonlinear, equal errors in latent space can produce different joint-space errors. We therefore augment the flow-matching loss with supervision on the decoded hand action. Let z be the latent target, $\epsilon \sim \mathcal{N}(0, I)$, and t_{flow} the flow time. We construct $x_{t_{\text{flow}}} = t_{\text{flow}}\epsilon + (1 - t_{\text{flow}})z$ and train the action expert $v_\omega(x_{t_{\text{flow}}}, t_{\text{flow}}, o)$ against $u = \epsilon - z$. Its one-step estimate of the clean chunk is $\hat{z} = x_{t_{\text{flow}}} - t_{\text{flow}} v_\omega$, whose hand blocks decode to $\hat{a}_{1:H}^{\text{hand}} =$

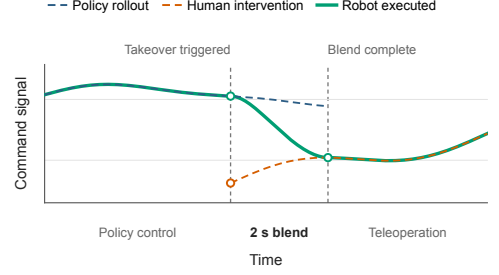


Figure 3. Smooth hand-command transition during a DAgger takeover. Dashed curves show the policy and teleoperation commands, while the solid curve shows the command executed by the robot. At takeover, the executed command transitions over 2 s from the last policy command to the live teleoperation command and remains continuous across the switch.

$D^{\text{hand}}(\hat{z}^{\text{hand}}, q_0^{\text{hand}})$. The objective is

$$\begin{aligned} \mathcal{L}_{\text{sft}} = \mathbb{E} [& \lambda_{\text{arm}} \|(v_\omega - u)^{\text{arm}}\|^2 \\ & + \lambda_{\text{lat}} \|(v_\omega - u)^{\text{hand}}\|^2] \quad (10) \\ & + \lambda_{\text{dec}} \mathbb{E} \|\hat{a}_{1:H}^{\text{hand}} - a_{1:H}^{\text{hand}}\|_2^2, \end{aligned}$$

Here, λ_{arm} weights flow matching on the 14 uncompressed arm-joint coordinates, λ_{lat} on the 18 hand-latent coordinates, and λ_{dec} on the decoded hand-action error. Gradients from the decoded-action loss are backpropagated through the frozen decoder D^{hand} to update ω .

Step 3: DAgger

Supervised fine-tuning visits mostly expert-induced states, whereas in closed-loop execution small errors in approach, finger contact, or object pose alter subsequent observations and drive the robot out of that distribution. We therefore aggregate operator corrections using human-gated DAgger (Ross et al. 2011; Kelly et al. 2019).

First, the last 5 s of robot state is held in a rolling buffer: when the operator judges that continued autonomous execution would fail or breach a safety boundary, they select a point inside that window and the robot

is returned to it, so the correction starts before the error compounded rather than at the moment it became visible. Second, an interface displays the operator’s pose alongside the robot’s to help the operator align the two before takeover. Once takeover is triggered, we smoothly blend the hand-joint command over 2s from the last command sent to the robot to the live teleoperation command, preventing a step change at the control switch. Let q^{hold} denote the last hand-joint command sent before takeover and q_t^{teleop} the live operator command. For elapsed blend time s and blend duration $\tau_b = 2\text{s}$, the hand command sent to the robot is the linear interpolation

$$q_t^{\text{hand,cmd}} = (1 - \alpha_s)q^{\text{hold}} + \alpha_s q_t^{\text{teleop}},$$

$$\alpha_s = \min\left\{\frac{s}{\tau_b}, 1\right\}. \quad (11)$$

so that $\alpha_s = 0$ at the start of takeover and $\alpha_s = 1$ after the blend completes. Figure 3 illustrates this transition. The operator then completes the episode.

For supervision, the live operator command is recorded throughout the blend rather than the blended command sent to the robot. The retained joint-space trajectory is converted to the representation of equation 1 and, for the latent-action policy, encoded with E . After each collection round, the resulting corrections are appended to the dataset, and the policy is retrained for 30k update steps from the same public $\pi_{0.5}$ checkpoint. Each minibatch draws a fraction α_{new} from the newly collected corrections and the remaining $1 - \alpha_{\text{new}}$ from the previously accumulated demonstrations and corrections.

Step 4: Real-world RL

The RL token. To represent the observation, we use the RL token of Xu et al. (2026), a single vector h_{rl} summarizing what the frozen backbone has computed about o . To obtain h_{rl} , we take the final-layer image-token features $\eta_{1:M}$ from a prefix-only pass through the frozen backbone, append a learnable readout

token ξ_{rl} , and process the resulting sequence with a two-layer bidirectional transformer g_φ . Each encoder block contains bidirectional self-attention, a second attention sublayer over the sequence entering that block, and a feed-forward sublayer. The output at the appended readout position $M + 1$ is then projected by a learned linear layer W_{rl} to obtain

$$h_{\text{rl}} = W_{\text{rl}} g_\varphi([\eta_{1:M}; \xi_{\text{rl}}])_{M+1} \in \mathbb{R}^{2048}. \quad (12)$$

The image-token features already incorporate the language instruction and proprioceptive state through the backbone’s bidirectional prefix attention. To retain this information in h_{rl} , we train the readout on the reference policy’s demonstration set $\mathcal{D}$ by reconstructing its image-token features with an auxiliary autoregressive decoder d_ψ , while keeping the backbone frozen. The decoder uses teacher forcing with right-shifted target features, causal self-attention, and cross-attention to h_{rl} . Over the unpadded image-token positions $\mathcal{V}$, we average the squared reconstruction error across positions and their 2048 feature coordinates:

$$\mathcal{L}_{\text{ro}}(\varphi, \psi) = \mathbb{E}_{o \sim \mathcal{D}} \left[\frac{1}{2048 |\mathcal{V}|} \sum_{i \in \mathcal{V}} \|d_\psi(h_{\text{rl}}, \eta_{<i}) - \eta_i\|_2^2 \right], \quad (13)$$

The prefix features are stop-gradiented, so only the readout token, encoder, projection, and auxiliary decoder receive gradient updates. After training, we discard d_ψ and freeze the readout parameters $(\xi_{\text{rl}}, g_\varphi, W_{\text{rl}})$. During online RL, the frozen readout reuses the reference policy’s prefix features to compute h_{rl} once per chunk. Implementation details and the default offline pretraining configuration are given in Appendix B.3 and Table 6.

The latent residual. Real-robot data is too scarce to continue training the resulting policy, and updating its parameters would eliminate it as an exact fallback. We therefore freeze the resulting policy, denoted by π_{ref} , and

learn a bounded residual Δz in the same space as its latent output $z_{\text{ref}} = \pi_{\text{ref}}(o)$. The corrected latent action $z_{\text{ref}} + \Delta z$ is passed through the frozen decoder D , so setting $\Delta z = 0$ exactly recovers the reference policy’s original command.

Let f_θ denote the MLP residual actor conditioned on h_{rl} , q , and z_{ref} . The actor produces a bounded per-frame correction c , from which we form the full-horizon residual Δz and the resulting joint-command chunk $q_{1:H}^{\text{cmd}}$:

$$c_{i,d} = \begin{cases} b_d^c \tanh\left([f_\theta(h_{\text{rl}}, q, z_{\text{ref}})]_{i,d}\right), & 1 \leq i \leq C, \\ 0, & C < i \leq H, \end{cases} \quad (14)$$

$$\Delta z_{i,d} = \begin{cases} \sum_{j=1}^i c_{j,d}, & \text{arm dimension } d, \\ c_{i,d}, & \text{hand-latent dimension } d \end{cases} \quad (15)$$

$$q_{1:H}^{\text{cmd}} = q + D(z_{\text{ref}} + \Delta z; q). \quad (16)$$

Here, q is broadcast across the H frames.

The bound b_d^c sets the allowed per-frame correction for each action-dimension group, with $b_d^c = b_{\text{arm}}^c$ for the arm dimensions and $b_d^c = b_{\text{hand}}^c$ for the hand-latent dimensions. The actor’s output layer is zero-initialised, so training starts at $\Delta z = 0$. We apply a task-specific residual mask before adding the correction to the reference action.

$$\Delta z \leftarrow m_{\text{task}} \odot \Delta z. \quad (17)$$

Here, $\odot$ denotes element-wise multiplication, and m_{task} is a binary mask over the residual action dimensions, broadcast across the H frames.

For each hand, the residual modifies nine latent coordinates per active frame, and the full latent sequence is decoded jointly into joint commands. This provides a compact, learned parameterisation of hand-motion corrections.

Chunk-level residual TD3. Each RL transition spans the C frames executed between two policy queries. At chunk t , the actor takes $h_{\text{rl},t}$, q_t , and $z_{\text{ref},t}$ as input and produces $c_{1:C}$, from which the full-horizon latent residual action Δz_t is formed as defined above. The resulting latent chunk $z_{\text{ref},t} + \Delta z_t$ is decoded and executed for C frames. After each episode, the operator labels the outcome; the reward assigned to its terminal chunk is $r_T = -1$ for failure and $r_T = 0$ for success, while all preceding chunks receive zero reward.

We optimise the residual with TD3 (Fujimoto et al. 2018), whose deterministic actor and twin critics operate at this chunk-level time scale. We maintain target copies of the actor and twin critics, denoted $f_{\bar{\theta}}$ and $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$, and update them by Polyak averaging. The twin critics Q_{ϕ_1} and Q_{ϕ_2} take $(h_{\text{rl}}, q, z_{\text{ref}} + \Delta z)$ as input and regress on the clipped double-Q target

$$y_t = r_t + (1 - d_t) \gamma^C \min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(h'_{\text{rl}}, q', z'_{\text{ref}} + \Delta z'), \quad (18)$$

Here, $d_t = 1$ for the terminal transition and $d_t = 0$ otherwise. The per-frame discount γ gives a discount of γ^C over each C -frame transition. Primes denote quantities at the next chunk, and $\Delta z'$ is formed using the target actor $f_{\bar{\theta}}$.

During collection, we add exploration noise to the bounded correction $c_{1:C}$, yielding $\tilde{c}_i = c_i + \sigma \varepsilon_i$, and form Δz from $\tilde{c}_{1:C}$. We use separate noise scales σ^{arm} and σ^{hand} for the arm and hand-latent dimensions, respectively. For exploration within a chunk, we add white noise to the arm increments and accumulate these increments across the chunk, whereas the hand latents receive per-dimension normalised Brownian noise directly, with spectral exponent $\nu = 2$ and a $1/f^2$ power spectrum (Eberhard et al. 2023). Before execution, the perturbed chunk passes through the downstream safety clamps; evaluation uses no exploration noise.

A replay transition stores $(h_{\text{rl}}, q, z_{\text{ref}}, \Delta z, r, d)$, together with the corresponding next-chunk quantities

$(h'_{\text{rl}}, q', z'_{\text{ref}})$. Here, the stored Δz is the task-masked residual formed during collection, including exploration noise. For critic updates, the sampled latent action is reconstructed as $z_{\text{ref}} + \Delta z$. We define the residual-magnitude regulariser on the task-masked, full-horizon latent residual as

$$\mathcal{R}_{\Delta z}(\Delta z) := \|\Delta z\|_F^2 = \sum_{i=1}^H \sum_{d=1}^{32} (\Delta z_{i,d})^2. \quad (19)$$

The norm covers all H frames, with inactive dimensions contributing nothing. Beyond C , hand-latent residuals are zero, while active arm residuals retain their final accumulated values and remain included in the regulariser. The critics regress on equation 18 at every step; once joint training is enabled, the actor is updated after every critic update, minimising

$$\begin{aligned} & -\beta_q Q_{\phi_1}(h_{\text{rl}}, q, z_{\text{ref}} + \Delta z) \\ & + \beta_{\Delta z} \mathcal{R}_{\Delta z}(\Delta z) \\ & + \beta_s \mathcal{S}(z_{\text{ref}} + \Delta z), \end{aligned} \quad (20)$$

For a latent chunk $z_{1:H}$, the temporal smoothness penalty is

$$\mathcal{S}(z_{1:H}) = \frac{1}{H-2} \sum_{i=1}^{H-2} \|z_{i+2} - 2z_{i+1} + z_i\|_2^2. \quad (21)$$

In equation 20, Δz is recomputed without exploration noise using the current actor and the stored $(h_{\text{rl}}, q, z_{\text{ref}})$, and $z_{1:H} = z_{\text{ref}} + \Delta z$. The three terms respectively maximise the critic value, penalise the residual magnitude through $\mathcal{R}_{\Delta z}$, and penalise the second temporal difference of this resulting latent chunk. The actor objective is evaluated entirely in the latent space, so its gradients do not pass through D . The loss weights and remaining RL hyperparameters are listed in Table 7 of Appendix B.

Online post-training protocol. Online post-training proceeds in two phases. During the *warm-up* phase, the actor output is set to zero, exploration noise is applied, and only

the critics train while the operator watches their value estimates on live rollouts. The operator manually starts the *joint training* phase with a keystroke, after confirming that critic training has stabilised and the value estimates distinguish success from failure. In this phase, the actor output is executed with exploration noise. The critics continue to train, and the actor is updated after every critic update.

Before decoding, we form the full H -frame residual Δz from the actor's C -frame output as in equation 15 and add it to z_{ref} . Three safety bounds are applied before execution. First, b^c bounds the per-frame correction c before exploration. Second, the hand latents in $z_{\text{ref}} + \Delta z$ are clipped to the 99.5th-percentile envelope b^z recorded during VAE training. Third, the final joint-position command is rate-limited to a maximum change of $\dot{q}_{\text{max}}$ per control frame.

4 Experiments

We evaluate the proposed post-training pipeline and its individual stages on five real-world dexterous manipulation tasks, testing each reported checkpoint over 20 trials. The final policy for every task succeeds in all 20 evaluation trials.

4.1 Experimental setup

Hardware and control. We use a bimanual platform with two 7-DoF arms from Tianji and two 20-DoF hands from Wuji (Figure 4). We use one head-mounted RGB camera and two wrist-mounted RGB cameras, with one wrist camera attached to each arm. We teleoperate the arms with a Vive Tracker and the hands using both Wuji Glove and Manus gloves. Both the arms and hands are controlled in joint position at 30 Hz. We record the three camera views, joint commands, and measured joint states at 30 Hz as well. The final joint-position command is rate-limited to a maximum change of $\pi/18$ rad per control frame.

Tasks. The five tasks illustrated in Figure 5 use the same marker and span grasping,

Algorithm 1 Online post-training with critic warm-up followed by joint actor–critic training.**Require:** frozen π_{ref} , decoder D , RL-token encoder g_ϕ **Require:** actor f_θ , critics Q_{ϕ_1}, Q_{ϕ_2} , buffer $\mathcal{B}$, phase $P \in \{\text{WARMUP}, \text{JOINT}\}$ initialised to WARMUP**Serving loop***▷ one iteration per action chunk*

```

1: for each episode do
2:   for  $t = 1, 2, \dots$  until the operator ends the episode do
3:     observe  $o_t = (I_{1:K}, \ell, q_t)$ 
4:      $h_{\text{rl}} \leftarrow$  readout of  $g_\phi$  on the prefix of  $\pi_{\text{ref}}(o_t)$  ▷ equation 12
5:      $z_{\text{ref}} \leftarrow \pi_{\text{ref}}(o_t) \in \mathbb{R}^{H \times 32}$  ▷ no encoding
6:     if  $P = \text{JOINT}$  then
7:        $c \leftarrow b^c \cdot \tanh(f_\theta(h_{\text{rl}}, q_t, z_{\text{ref}}))$  ▷ equation 14
8:     else
9:        $c \leftarrow 0$  ▷ actor output disabled
10:    end if
11:     $c \leftarrow c + \sigma\epsilon$  ▷ white arm noise on increments; direct Brownian  $\nu=2$  hand noise
12:    form task-masked  $\Delta z$  from  $c$  using equation 15 and equation 17
13:     $z \leftarrow z_{\text{ref}} + \Delta z$ ; clamp  $z^{\text{hand}}$  to  $b^z$ 
14:     $q_{1:H}^{\text{cmd}} \leftarrow q_t + D(z; q_t)$  ▷  $q_t$  broadcast over  $H$  frames
15:    rate-limit  $q_{1:H}^{\text{cmd}}$  to a maximum change of  $\dot{q}_{\text{max}}$  per control frame
16:    execute the first  $C$  frames; record the transition ▷ store  $z_{\text{ref}}$  and  $\Delta z$  separately
17:  end for
18:  operator labels the episode outcome
19:  assign  $r = -1$  to the terminal transition on failure and  $r = 0$  to all other transitions; append them to  $\mathcal{B}$ 
20: end for

```

Learner*▷ runs once $|\mathcal{B}|$ exceeds a minimum size*

```

21: while post-training is running do
22:   sample a batch from  $\mathcal{B}$ 
23:   update  $\phi_1, \phi_2$  on the target  $y$  of equation 18 ▷ critic action: stored  $z_{\text{ref}} + \Delta z$ 
24:   if  $P = \text{JOINT}$  then
25:     recompute  $\Delta z$  using the current actor without exploration noise
26:     after each critic update, update  $\theta$  on equation 20 and export a snapshot
27:   end if
28:   Polyak-update the target critic; if  $P = \text{JOINT}$ , also update the target actor
29: end while

```

bimanual transfer, in-hand manipulation, object separation, and contact-rich writing. Each task uses an operator-checked success criterion. Videos of these tasks are available on the [project website](#).

- *Pick Up Marker (Pick up)* succeeds when the robot lifts the marker completely off the table and ends with the marker securely grasped.
- *Transfer Marker between Hands (Transfer)* succeeds when the marker is

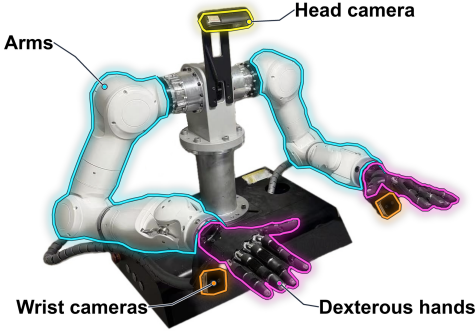


Figure 4. Robot platform used for data collection and evaluation. The system comprises two 7-DoF Tianji arms, two 20-DoF Wuji dexterous hands, one RGB wrist camera mounted on each arm, and one RGB head camera.

passed from the right hand into a stable grasp in the left hand and is then secured between the right index and middle fingers.

- *Rotate Marker in Hand (Rotate)* succeeds when the right hand rotates the marker without dropping it and finishes with its tip pointing downward.
- *Remove Marker Cap (Uncap)* succeeds when the left hand completely separates the cap from the marker without dropping either component.
- *Write on Paper (Write)* succeeds when the robot writes the complete word “WUJI,” the brand name of the dexterous hands used in our setup, while maintaining contact between the marker tip and the paper.

Across these tasks, task-specific adaptation begins with supervised fine-tuning. Pick Up Marker, Remove Marker Cap, and Write on Paper reach 100% success after DAGger. Transfer Marker between Hands requires DAGger followed by latent residual RL, whereas Rotate Marker in Hand proceeds directly from supervised fine-tuning to latent residual RL because its physical object state cannot be restored during takeover.

Every trial is reset manually. Pick Up Marker starts with the marker lying on the right half of the table. Transfer Marker between Hands starts with the marker grasped in the right hand. Rotate Marker in Hand starts with the marker held between the right index and middle fingers. Remove Marker Cap starts with the capped marker already held by the robot. Write on Paper starts with the marker already held by the robot and a sheet of paper placed at the centre of the table. Each checkpoint is evaluated over 20 task trials under the corresponding task-specific reset condition, and its success rate is the fraction of successful trials. Training random-seed settings are held fixed across the dataset comparisons, while the training data vary with the collection stage and budget.

Lighting is held approximately constant throughout evaluation, while for each task, the initial object pose or grasp is manually varied within the range represented in the data-collection distribution.

4.2 Training the hand action codec

Training corpus. The codec is trained on three data sources. First, our internal teleoperation corpus contains approximately 38M frames per hand, collected across robots in the same family and a range of manipulation tasks. Second, we use the VITRA-1M human-hand trajectories reconstructed from egocentric video and retargeted to our 20-DoF hand (Li et al. 2026a). Third, we include the task-specific demonstration set used in Step 2, with 200 episodes per task. The task-specific codec data are restricted to these initial 200 demonstrations per task and exclude all subsequent DAGger corrections and online RL data. We train separate codecs for the two hands on the combined corpus. Full optimisation and loss settings are provided in Appendix B.

Comparison of compression methods. To assess how much of the hand’s motion each representation can preserve, we compare the reconstruction quality of several compression methods, all evaluated at nine latent dimensions

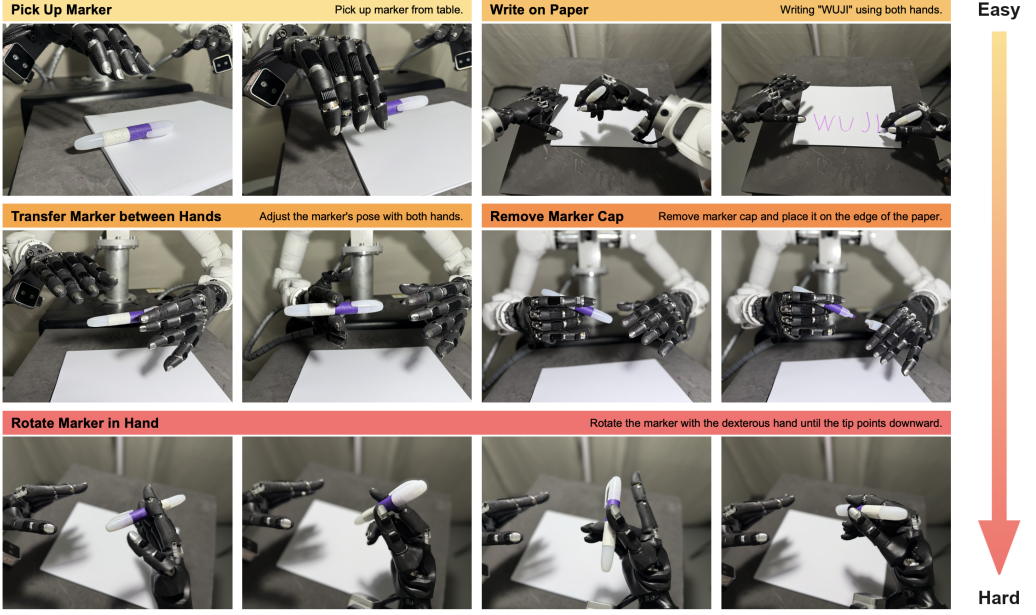


Figure 5. The five marker-manipulation tasks used for evaluation: Pick Up Marker, Transfer Marker between Hands, Rotate Marker in Hand, Remove Marker Cap, and Write on Paper. The tasks cover grasping, bimanual transfer, in-hand manipulation, object separation, and contact-rich writing. The yellow-to-red arrow orders the tasks from easier to harder.

per hand. The methods differ in linearity, temporal context, and architecture. Principal component analysis (PCA) provides a frame-wise linear baseline motivated by postural synergies in hand configurations (Santello et al. 1998). It independently projects each standardised 20-dimensional hand action onto nine principal components and reconstructs it without modelling dependencies across frames. Two frame-based autoencoders — an MLP and a Transformer-based VAE — encode each frame in isolation and therefore carry no temporal context. The remaining two operate on whole chunks: a chunk-based MLP, which receives the chunk flattened into a single vector, and our chunk-level variational Transformer, which attends across the H frames of a chunk and serves as the codec of [Step 1](#).

All methods are trained on the same corpus and evaluated on the same held-out validation set. We report the reconstruction error in

degrees per joint, averaged over the 40 joints of both hands, in [Figure 6\(a\)](#).

Choosing the latent dimensionality. With the architecture fixed, we sweep the latent width of the chunk-level Transformer and measure the reconstruction error on the same held-out validation set, reported in [Figure 6\(b\)](#). Two considerations set the operating point. The first is accuracy, which improves monotonically with width but at a diminishing rate: the three dimensions from six to nine remove about 1.4 times as much error as the three from nine to twelve do, so each further dimension buys less reconstruction quality than the one before it.

The second is a constraint from the reference policy: at nine dimensions per hand the layout of [equation 8](#) totals $2 \times 7 + 2 \times 9 = 32$ dimensions, which is the largest action dimensionality the pretrained $\pi_{0.5}$ checkpoint accepts. Nine dimensions per hand meets both,

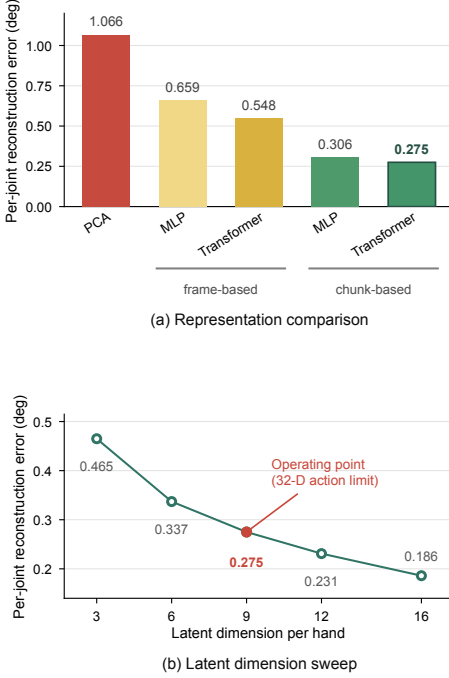


Figure 6. Hand-action codec analysis. Panel (a) compares the reconstruction error of five compression methods, all using nine latent dimensions per hand. Panel (b) shows the reconstruction error of the chunk-level Transformer codec as the latent width varies. All errors are measured on the same held-out validation set, reported in degrees per joint, and pooled across the 40 joints of both hands. The nine-dimensional operating point is selected to satisfy the pretrained policy’s 32-dimensional action interface.

and is the width used in every experiment reported here.

4.3 Supervised fine-tuning and data aggregation

We first supervised fine-tune separate policies on 50, 100, 150, 200, 250 and 300 demonstrations, each for 30 k update steps. Using the 200-demonstration policy for the initial collection, we then collect two rounds of 50 corrections using the takeover procedure of [Step 3](#), with $\alpha_{\text{new}} = 0.5$. After each round, we reinitialise the VLA from the same public $\pi_{0.5}$ checkpoint

and train for 30 k update steps on the aggregated dataset. All policies use a batch size of 64; [Appendix B](#) gives the remaining optimisation details. We evaluate DAgger on the four tasks that admit takeover; Rotate Marker in Hand is evaluated under SFT only.

For the continued-imitation controls in [Figure 7](#), we additionally train demonstration-only policies on 350, 400 and 450 demonstrations for Transfer Marker between Hands and on 350 demonstrations for Rotate Marker in Hand. The continued-DAgger baseline for Transfer Marker between Hands adds three further rounds of 50 corrections after the 300-episode checkpoint, yielding 350, 400 and 450 cumulative episodes. Each additional correction round uses the policy trained on the preceding aggregated dataset for collection. All continued-imitation checkpoints follow the same 30 k-update training schedule from the public $\pi_{0.5}$ checkpoint, with $\alpha_{\text{new}} = 0.5$ for DAgger rounds and no online RL. The three analyses below build on this experimental setup.

1) Are DAgger corrections more sample-efficient than additional demonstrations?

[Figure 7](#) compares each DAgger checkpoint with a demonstration-only policy trained from the same cumulative episode budget. At both 250 and 300 episodes, the DAgger pipeline achieves higher success across the four tasks that admit takeover.

Additional demonstrations are collected independently of the current policy and therefore do not adapt to its changing failure modes. DAgger instead uses rollouts of the current policy to identify where corrections are needed. At each intervention, the operator provides a recovery trajectory from a buffered pre-failure state, so successive rounds focus on the errors that remain.

2) Does imitation in the hand-latent space preserve raw joint-space performance?

To isolate the effect of the hand-latent representation, we train a raw-action baseline by expanding the action head from 32 to 54 dimensions, allowing it to predict joint-space

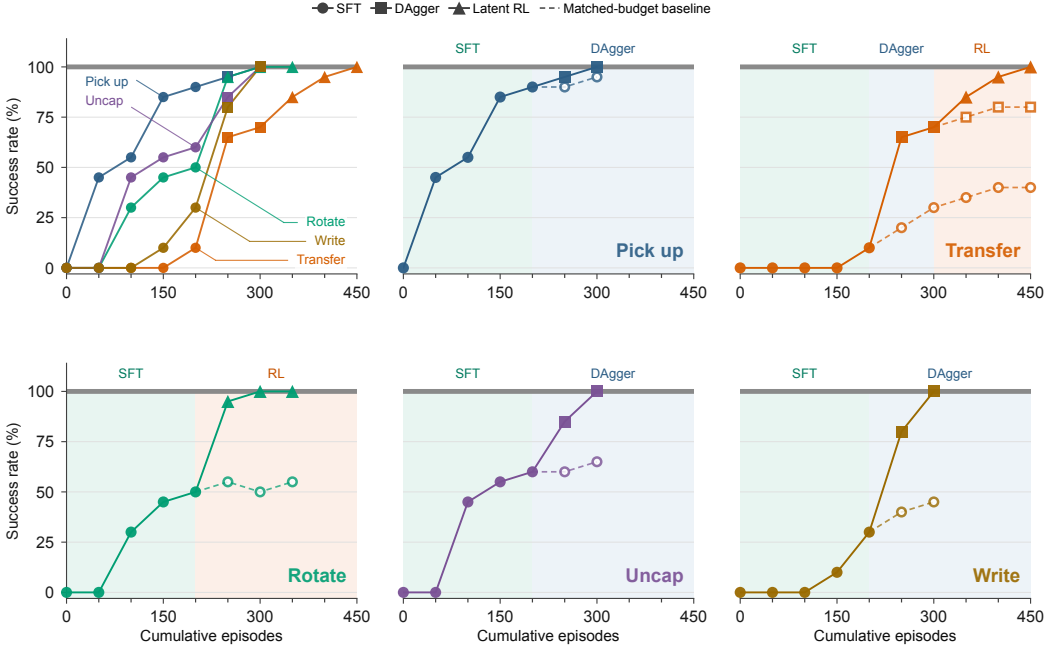


Figure 7. Success rate as a function of cumulative episodes collected. The overview panel overlays the trajectories for all five tasks, labelled directly on their curves; the remaining panels show the tasks individually. Marker shapes identify training methods: circles for SFT, squares for DAgger, and triangles for latent residual RL. Solid lines with filled markers show our staged pipeline; dashed lines with open markers show matched-budget baselines that continue the indicated method. Background shading marks the stage reached by the main pipeline. Each checkpoint is evaluated over 20 trials. Rotate Marker in Hand admits no corrections and moves directly from SFT to online RL.

actions directly without the codec. Both variants use the same SFT demonstration subsets and a matched DAgger budget of two 50-correction rounds from their respective 200-demonstration checkpoints. All other training and evaluation settings are held fixed. Their success-rate curves remain closely aligned, showing no consistent loss in imitation performance from compressing each hand from 20 to 9 dimensions (Figure 8).

3) Where does imitation learning remain insufficient?

Data aggregation does not close the gap on every task, and on Transfer Marker between Hands, Remove Marker Cap, and Write on Paper the gain from the second 50-episode correction round is smaller than from the first, indicating that improvement slows as

aggregation proceeds; Pick Up Marker is already near saturation after SFT. Transfer Marker between Hands rises from 10% to 65% after the first 50 corrections and to 70% after the second round, but remains short of full success. Rotate Marker in Hand reaches 50% under SFT and cannot use DAgger because its physical object state cannot be restored at takeover. These remaining gaps motivate the online residual RL evaluated in Section 4.4.

4.4 Online residual reinforcement learning

Online post-training is evaluated on Transfer Marker between Hands and Rotate Marker in Hand, the two tasks left below full success after imitation learning. Transfer Marker between Hands starts from the 70% DAgger checkpoint,

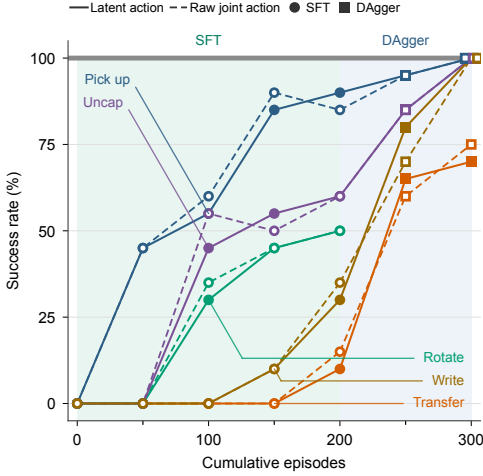


Figure 8. Imitation learning in the raw joint space and the learned hand-latent space across the five tasks.

whereas Rotate Marker in Hand starts from the 50% SFT checkpoint. For Transfer Marker between Hands, the task-specific mask in equation 17 selects the full 32-dimensional latent action, comprising both 7-DoF arms and both 9-dimensional hand latents. For Rotate Marker in Hand, the mask selects only the 9-dimensional latent of the active right hand and fixes every arm and left-hand residual dimension to zero. The two analyses below use these settings.

1) How do latent and raw-space residual RL compare with the frozen policy?

To isolate the effect of the residual action space, we compare the frozen policy, a raw-space residual policy (Johannink et al. 2019), and the latent-residual policy of Step 4. The raw-space actor adds its residual directly to the joint-position action chunk produced by the frozen VLA decoder, whereas the latent-space actor adds its residual to z_{ref} before the same frozen decoder. Specifically, letting $a_{\text{ref}} = D(z_{\text{ref}}; q)$ denote the decoded reference action, the raw-space actor takes $h_{\text{rl}} \in \mathbb{R}^{2048}$, $q \in \mathbb{R}^{54}$, and $a_{\text{ref},1:C} \in \mathbb{R}^{C \times 54}$ as input and outputs a

$C \times 54$ joint-space residual:

$$\begin{aligned} \Delta_{1:C}^{\text{raw}} &= b^{\text{raw}} \odot \tanh(f_{\psi}^{\text{raw}}(h_{\text{rl}}, q, a_{\text{ref},1:C})) \\ &\in \mathbb{R}^{C \times 54}, \\ q_{1:C}^{\text{cmd,raw}} &= q + a_{\text{ref},1:C} + \Delta_{1:C}^{\text{raw}}. \end{aligned} \quad (22)$$

Thus, the raw actor outputs a bounded residual in the decoded joint-action space, with no hand-latent decoding in the residual path. Both variants otherwise use the same TD3 training and execution protocol, online interaction budget, task-specific residual scope, and safety limits. They also use the same temporal exploration-noise construction: white noise is added to the arm increments and accumulated across the chunk, while direct per-dimension Brownian noise with $\nu = 2$ is applied to the hand block. The noise amplitudes are chosen to match the magnitude of the resulting joint-space perturbations, measured after decoding for the latent variant (Appendix B).

Figure 9 follows both residual methods over 3000 online transitions. At the 30 Hz control rate, each transition spans $C = 25$ executed frames, or approximately 0.83 s of commanded motion. The latent-residual policy improves Transfer Marker between Hands from 70% to 100% and Rotate Marker in Hand from 50% to 100%. See the videos on the [project website](#) for the post-trained VLA executing both tasks. The raw-residual policy ends at 80% and 50%, respectively, and its success rate fluctuates rather than increasing consistently over training. Qualitatively, under exploration, its joint-space perturbations often appear as uncoordinated motion across individual joints rather than a coherent corrective action.

The performance gap between the two residual policies can be understood by examining how their action spaces shape exploration. The raw residual can perturb the active joint coordinates independently, whereas the latent residual is decoded from a hand representation learned from recorded motion. The latent action space may therefore bias exploration toward coordinated hand motion and reduce

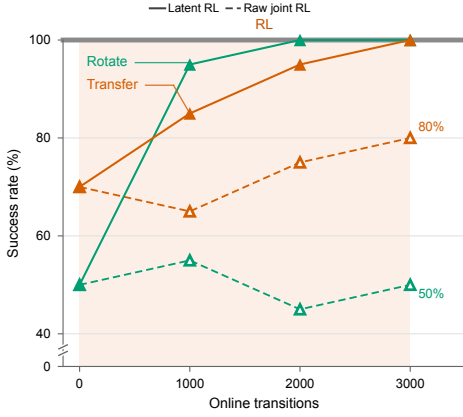


Figure 9. Comparison of latent residual RL and raw-space residual RL over online interaction.

the unstructured joint-space directions that must be explored online. The contrast is clearest on Rotate Marker in Hand, where the correction acts entirely on the hand.

Across multiple evaluation trials, the latent-residual policy produces corrective behaviours not observed in our frozen-policy trials, including regrasping, maintaining contact, and recovering from incipient errors. Their recurrence suggests that the improvement reflects learned corrective responses rather than isolated favourable perturbations. Additional trials with randomized initial positions and external disturbances applied to the object and robot also provide evidence of robustness beyond the nominal evaluation condition. Within these two tasks and the tested interaction budget, latent residual RL reaches full success, whereas raw-space residual RL does not.

2) *How efficiently does latent residual RL close the remaining imitation gap?*

Sample efficiency. Figure 7 compares SFT, DAGger, and latent residual RL on a common cumulative-episode axis for Transfer Marker between Hands and Rotate Marker in Hand. For the RL curves, 1000 online transitions correspond to 50 rollout episodes. The latent-residual policy surpasses the frozen policy at

that first evaluation and reaches 100% after 100 episodes on Rotate Marker in Hand and 150 on Transfer Marker between Hands.

Operator-time efficiency. Demonstrations require continuous teleoperation, DAGger requires rollout monitoring and occasional takeover, whereas RL uses autonomous rollouts with terminal outcome labelling. Table 2 reports the estimated operator time required to collect 100 episodes with each method, including manual resets and rollout monitoring.

Using the full pipeline, we obtain 100%-success policies for Transfer Marker between Hands and Rotate Marker in Hand with 9.7 and 5.2 worker-hours of data collection, respectively. Under the same collection-time budgets, the corresponding no-RL baselines reach only approximately 75% with SFT+DAGger and 55% with SFT, respectively. Taken together, these results show a favourable speed–performance trade-off for online latent residual RL: it requires only 1.2 worker-hours per 100 rollout episodes, yet reaches higher final success under the same collection-time budget than imitation-only post-training. Thus, online RL is both more operator-time efficient and more effective at closing the remaining performance gap, rather than merely reducing the cost of data collection.

4.5 Ablations

1) *How much does each component contribute?*

Table 2. Estimated human labor required to collect 100 episodes with SFT, DAGger, and online RL. The estimates use the normalized collection rates described in the text and should be interpreted as approximate operator-time comparisons rather than direct wall-clock measurements.

Collection method	Approx. worker-hours
SFT demonstrations	2.0
DAGger corrections	3.9
Online RL rollouts	1.2

We ablate five design choices on both online RL tasks, changing one at a time while keeping the rollout budget, safety limits, and optimisation settings fixed. Figure 10 compares the full method with a frame-wise 9-D PCA representation, temporally independent white exploration noise, removal of the residual regulariser $\mathcal{R}_{\Delta z}$, removal of the temporal smoothness penalty, and removal of critic warm-up. For the PCA variant, both compression and reconstruction operate independently on each frame, without temporal modelling or conditioning on the initial hand state. We retrain the reference policy in the PCA action space before freezing it for online residual RL. The residual actor retains its chunk-wise formulation, exploration noise, and regularisation; the absence of temporal modelling applies only to the PCA representation.

All variants underperform the full method. The PCA result favours the learned chunk-level representation over frame-wise linear compression under the reported settings. The representations differ in nonlinearity, temporal context, and initial-hand-state conditioning, and the PCA variant uses a retrained reference policy. Using temporally independent white noise weakens exploration because its frame-to-frame perturbations do not persist as coherent chunk-level corrections. Without the residual regulariser $\mathcal{R}_{\Delta z}$, the actor can deviate further from the reference latent action, potentially destabilising learning. Removing the temporal smoothness penalty can yield less smooth latent action sequences and reduce control quality. Without critic warm-up, actor updates rely on potentially unreliable early value estimates, which can slow and destabilise online improvement.

5 Limitations and Discussion

We observe that the effectiveness of residual RL depends strongly on the competence of the frozen reference policy. When the SFT or DAGger policy succeeds in fewer than

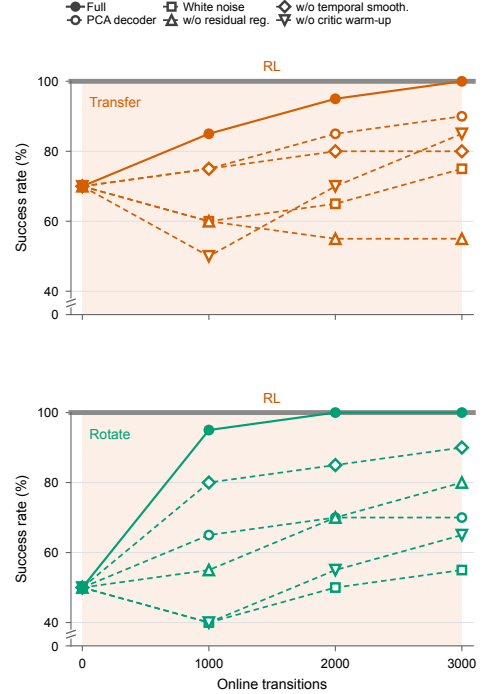


Figure 10. Ablation of five latent residual RL design choices on Transfer Marker between Hands and Rotate Marker in Hand. Each variant changes one component while retaining the remaining optimisation settings. Curves report measured success rates.

approximately 20% of evaluation trials, residual RL yields little measurable improvement, whereas it becomes consistently useful once the base-policy success rate exceeds approximately 50%. This observation suggests that the reference policy must already visit task-relevant states often enough for local residual exploration to obtain informative experience. We have not systematically characterized the intermediate regime or established that 50% is a universal threshold across tasks, objects, or embodiments. Residual RL should therefore be viewed as a late-stage improvement mechanism rather than a substitute for obtaining a minimally competent imitation policy.

The hand-latent dimensionality is also partly constrained by the action interface of the pretrained $\pi_{0.5}$ checkpoint rather than selected solely for optimal RL performance. Its 32-dimensional action interface allocates 14 dimensions to the two 7-DoF arms, leaving 18 dimensions for the two hands and therefore at most nine latent dimensions per hand under our symmetric layout. Although the codec sweep characterizes reconstruction quality at several latent widths, we have not decoupled this interface constraint from the effect of latent dimensionality on online RL. Consequently, our results do not establish that nine dimensions per hand is optimal for other foundation policies, embodiments, or interaction budgets; evaluating latent residual learning with independently variable action capacity remains an important direction for future work.

Finally, the pipeline is designed to improve a specified atomic dexterous skill that completes within approximately 30 s. The experiments do not address long-horizon skill composition or scene-level generalization, and the reported evaluations use local variations around the data-collection conditions rather than substantial changes in lighting, objects, or initial-state distributions. Extending the method from reliable task-specific post-training to policies that retain robustness across broader scenes and compose multiple dexterous skills remains future work.

6 Conclusion

We presented a staged post-training pipeline for adapting pretrained vision–language–action policies to real-world dexterous manipulation. The pipeline combines supervised fine-tuning, corrective data aggregation, and off-policy residual reinforcement learning through a shared hand-action representation learned from coordinated motion. Its temporally windowed takeover procedure enables DAGger corrections while retaining absolute hand commands, and its latent residual policy restricts online

exploration to a compact space of coordinated hand actions.

Across five marker-manipulation tasks, the successive post-training stages produced final policies that succeeded in all 20 evaluation trials for every task. Compressing each 20-dimensional hand command to a 9-dimensional latent preserved imitation-learning performance, while latent residual RL improved Transfer Marker between Hands from 70% to 100% and Rotate Marker in Hand from 50% to 100% within 3,000 real-world transitions. Under the same interaction budget, residual search in the raw joint space did not obtain the same improvements. These results support learned action representations as a practical interface between imitation learning and sample-efficient online improvement for high-degree-of-freedom hands. See the videos on the [project website](#) for the resulting real-world behaviors.

ORCID iDs

Junlei Zhu: [0009-0006-6518-3575](#)
 Shenzhe Yao: [0009-0006-0016-5501](#)
 Chaogui Huang: [0009-0007-3097-1667](#)
 Wenkai Zhu: [0009-0002-1928-5934](#)
 Jingwei Peng: [0009-0006-9428-6004](#)
 Guanqi He: [0009-0007-6016-9800](#)
 Sören Schwertfeger: [0000-0003-2879-1636](#)
 Jiahao Chen: [0000-0002-8927-5646](#)
 Yide Liu: [0000-0002-2447-3107](#)

Declaration of conflicting interests

The authors declare that they have no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Supplemental material

Supplemental material for this article is available online

References

- Allshire A, Martín-Martín R, Lin C, Manuel S, Savarese S and Garg A (2021) LASER: Learning a latent action space for efficient reinforcement learning. *arXiv preprint arXiv:2103.15793*.
- Amin A, Aniceto R, Balakrishna A, Black K et al. (2025) $\pi_{0.6}^*$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*.
- Black K, Brown N, Driess D, Esmail A, Equi M, Finn C, Fusai N, Groom L, Hausman K, Ichter B, Jakubczak S, Jones T, Ke L, Levine S, Li-Bell A, Mothukuri M, Nair S, Pertsch K, Shi LX, Tanner J, Vuong Q, Walling A, Wang H and Zhilinsky U (2024) π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*.
- Brohan A, Brown N, Carbajal J, Chebotar Y, Chen X, Choromanski K, Ding T, Driess D, Finn C, Hausman K, Ichter B, Levine S et al. (2023) RT-2: Vision-language-action models transfer web knowledge to robotic control. In: *Conference on Robot Learning (CoRL)*, volume 229. pp. 2165–2183. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.
- Chen Z, Han Y, Shao Y, Liu H, Xu C, Chen X, Mu Y and Lian W (2026) BORA: Bridging offline reinforcement learning and online residual adaptation for real-world dexterous VLA models. *arXiv preprint arXiv:2605.30226*.
- Eberhard O, Hollenstein J, Pinneri C and Martius G (2023) Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In: *International Conference on Learning Representations (ICLR)*.
- Fujimoto S, van Hoof H and Meger D (2018) Addressing function approximation error in actor-critic methods. In: *International Conference on Machine Learning (ICML)*.
- Han Y, Chen Z, Zhao Y, Xu C, Shao Y, Peng Y, Mu Y and Lian W (2026) DexHiL: A human-in-the-loop framework for vision-language-action model post-training in dexterous manipulation. *arXiv preprint arXiv:2603.09121*.
- He Z and Ciocarlie M (2021) Discovering synergies for robot manipulation with multi-task reinforcement learning. In: *Conference on Robot Learning (CoRL)*.
- Huber PJ (1964) Robust estimation of a location parameter. *The Annals of Mathematical Statistics* 35(1): 73–101. DOI:10.1214/aoms/1177703732.
- Johannink T, Bahl S, Nair A, Luo J, Kumar A, Loskyll M, Ojea JA, Solowjow E and Levine S (2019) Residual reinforcement learning for robot control. In: *IEEE International Conference on Robotics and Automation (ICRA)*. pp. 6023–6029. DOI:10.1109/ICRA.2019.8794127.
- Kelly M, Sidrane C, Driggs-Campbell K and Kochenderfer MJ (2019) HG-DAGger: Interactive imitation learning with human experts. In: *IEEE International Conference on Robotics and Automation (ICRA)*.
- Kingma DP and Welling M (2014) Auto-encoding variational Bayes. In: *International Conference on Learning Representations (ICLR)*.
- Li Q, Deng Y, Liang Y, Luo L, Zhou L, Yao C, Zeng L, Feng Z, Liang H, Xu S, Zhang Y, Chen X, Chen H, Sun L, Chen D, Yang J and Guo B (2026a) Scalable vision-language-action model pretraining for robotic manipulation with real-life human activity videos. In: *IEEE International Conference on Robotics and Automation (ICRA)*.
- Li Z, Huang L, Xu W, Zhu Z, Lin N, Ma X, Sheng X and Wen R (2026b) Hand-in-the-loop: Improving VLA policies for dexterous manipulation via seamless hand–arm intervention. *arXiv preprint arXiv:2605.15157*.
- Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, Silver D and Wierstra D (2016) Continuous control with deep reinforcement learning. In: *International Conference on Learning Representations (ICLR)*.
- Lipman Y, Chen RTQ, Ben-Hamu H, Nickel M and Le M (2023) Flow matching for generative modeling. In: *International Conference on Learning Representations (ICLR)*.

- Liu H, Nasiriany S, Zhang L, Bao Z and Zhu Y (2025a) Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. *The International Journal of Robotics Research* 44(10–11): 1727–1742. DOI:10.1177/02783649241273901.
- Liu Y, Hamid JI, Xie A, Lee Y, Du M and Finn C (2025b) Bidirectional decoding: Improving action chunking via closed-loop resampling. In: *International Conference on Learning Representations (ICLR)*.
- Octo Model Team, Ghosh D, Walke H, Pertsch K, Black K, Mees O, Dasari S, Hejna J, Kreiman T, Xu C, Luo J, Tan YL, Chen LY, Sanketi P, Vuong Q, Xiao T, Sadigh D, Finn C and Levine S (2024) Octo: An open-source generalist robot policy. In: *Robotics: Science and Systems (RSS)*. URL <https://arxiv.org/abs/2405.12213>.
- Open X-Embodiment Collaboration (2024) Open X-embodiment: Robotic learning datasets and RT-X models. In: *IEEE International Conference on Robotics and Automation (ICRA)*. URL <https://arxiv.org/abs/2310.08864>.
- OpenAI, Akkaya I, Andrychowicz M, Chociej M, Litwin M, McGrew B, Petron A, Paino A, Plappert M, Powell G, Ribas R, Schneider J, Tezak N, Tworek J, Welinder P, Weng L, Yuan Q, Zaremba W and Zhang L (2019) Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:1910.07113*.
- OpenAI, Andrychowicz M, Baker B, Chociej M, Jozefowicz R, McGrew B, Pachocki J, Petron A, Plappert M, Powell G, Ray A, Schneider J, Sidor S, Tobin J, Welinder P, Weng L and Zaremba W (2020) Learning dexterous in-hand manipulation. *The International Journal of Robotics Research* 39(1): 3–20.
- Physical Intelligence, Black K, Brown N, Darpinian J, Dhabalia K, Driess D, Esmail A, Equi M, Finn C, Fusai N, Galliker MY, Ghosh D, Groom L, Hausman K, Ichter B, Jakubczak S, Jones T, Ke L, LeBlanc D, Levine S, Li-Bell A, Mothukuri M, Nair S, Pertsch K, Ren AZ, Shi LX, Smith L, Springenberg JT, Stachowicz K, Tanner J, Vuong Q, Walke H, Walling A, Wang H, Yu L and Zhilinsky U (2025) $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*.
- Rajeswaran A, Kumar V, Gupta A, Vezzani G, Schulman J, Todorov E and Levine S (2018) Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In: *Robotics: Science and Systems (RSS)*.
- Ross S, Gordon G and Bagnell D (2011) A reduction of imitation learning and structured prediction to no-regret online learning. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, Proceedings of Machine Learning Research*, volume 15. PMLR, pp. 627–635. URL <https://proceedings.mlr.press/v15/ross11a.html>.
- Santello M, Flanders M and Soechting JF (1998) Postural hand synergies for tool use. *Journal of Neuroscience* 18(23): 10105–10115.
- Wagenmaker A, Nakamoto M, Zhang Y, Park S, Yagoub W, Nagabandi A, Gupta A and Levine S (2025) Steering your diffusion policy with latent space reinforcement learning. In: *Conference on Robot Learning (CoRL)*.
- Xu C, Springenberg JT, Equi M, Amin A, Esmail A, Levine S and Ke L (2026) RL token: Bootstrapping online RL with vision-language-action models. *arXiv preprint arXiv:2604.23073*.
- Yu D, Lei K, Jiang Z, Pan J and Xu H (2026) Beyond action residuals: Real-world robot policy steering via bottleneck latent reinforcement learning. *arXiv preprint arXiv:2605.19919*.
- Zhao TZ, Kumar V, Levine S and Finn C (2023) Learning fine-grained bimanual manipulation with low-cost hardware. In: *Robotics: Science and Systems (RSS)*.
- Zhao Y, Bogdanovic M, Sohal A, Tao L, Darvish K, Aspuru-Guzik A, Shkurti F and Garg A (2026) Dynamic execution horizon prediction for chunk-based robot policies. *arXiv preprint arXiv:2606.11408*.
- Zhou W, Bajracharya S and Held D (2021) PLAS: Latent action space for offline reinforcement

learning. In: *Proceedings of the Conference on Robot Learning, Proceedings of Machine Learning Research*, volume 155. pp. 1719–1735.

A Architecture and parameter budget

Table 3 lists every module the system runs, its architecture and its exact parameter count, together with whether it is trained, frozen or discarded at each stage.

B Training schedules

B.1 Hand-action codec

Table 4 summarises the architectures used in the hand-representation comparison of Figure 6. Each method uses a nine-dimensional latent representation per frame.

Separate models are fitted for the left and right hands. All methods use the same episode-level 90/10 training–validation split with seed 0, with every frame from an episode assigned to the same partition; failed episodes are retained. The same per-joint standardisation procedure is applied using statistics computed from the training split only. Neural checkpoints are selected by the lowest validation per-joint MAE in the original joint units. PCA is fitted to the standardised training frames using nine components and evaluated on the same held-out episodes. The projection and reconstruction are applied independently to each frame, using a fixed basis for each hand. PCA uses neither temporal context nor conditioning on the initial hand state q_0 , and is fitted without a temporal training objective.

The frame-based neural models are trained for 200 epochs with AdamW, a batch size of 4096, cosine learning-rate decay from 10^{-3} , and weight decay 10^{-4} . They use the same weighted Huber reconstruction loss; the frame Transformer additionally uses a KL weight of 10^{-3} , linearly warmed up over the first 2000 steps. The chunk-based models are trained for 30k steps with AdamW, a batch size of 4096, cosine decay from 3×10^{-4} , and weight decay 10^{-4} . In equation 4, the Huber threshold is $\tau_{\text{Huber}} = 1$; the thumb, index, and middle-finger joints have weight 1, while the ring- and little-finger joints have weight 0.5; and

$(\lambda_0, \lambda_1, \lambda_2) = (1, 0.2, 0.05)$. The KL weight β_{KL} is zero for the first 4000 steps, increases linearly to 3×10^{-4} over the next 4000 steps, and remains constant thereafter.

B.2 Reference-policy training

Table 5 gives the training settings for the SFT and DAgger checkpoints in Figure 7. Within each stage, these settings are shared across tasks and dataset sizes; the latent and raw variants differ in the space used for flow matching, and the raw variant omits the decoded channel of equation 10, which has no meaning without the codec. Only the arm and hand weights are then in force, at λ_{arm} on all 54 dimensions.

The aggregation runs of Figure 7 reuse the same optimisation schedule. After all 50 corrections in a round have been collected, the VLA is reinitialised from the same public $\pi_{0.5}$ checkpoint and trained for 30k update steps; a fraction α_{new} of each minibatch is drawn from the newly collected corrections and the remainder from the previously accumulated demonstrations and corrections. We use $\alpha_{\text{new}} = 0.5$ in all aggregation runs.

Table 3. Architecture and parameter budget of every module in our system. Parameter counts are exact, obtained by traversing the Flax parameter tree of each module under its deployed configuration. The VLA operates on a 32-D latent action space (2×7 arm joints + 2×9 hand latents) with a prediction horizon $H=32$; the residual RL policy acts on chunks of length $C=25$.

Component	Architecture	Key configuration	#Params	Status
<i>(a) Hand latent codec: per-hand chunk VAE, 20-D $\rightarrow$ 9-D per step</i>				
Encoder (per hand)	Time-token Transformer (bidir.)	$d=128, L=3, h=4, \text{MLP} \times 2$	0.43 M	frozen
Decoder (per hand)	Time-token Transformer (bidir.)	$d=128, L=3, h=4, \text{MLP} \times 2$	0.43 M	frozen
<i>Subtotal, left + right, independent weights</i>			1.70 M	
<i>(b) Base VLA: $\pi_{0.5}$ fine-tuned to emit native 32-D latent actions</i>				
Vision encoder	SigLIP-So400m/14 ViT	$d=1152, L=27, h=16, 224^2$, 3 cams	414.80 M	frozen [†]
VLM backbone	Gemma-2B (MQA)	$d=2048, L=18, h=8/1$ KV, $d_{\text{ff}}=16384, N_{\text{vocab}}=257,152$	2.51 B	frozen [†]
Action expert	Gemma-300M + adaRMSNorm	$d=1024, L=18, h=8/1$ KV, $d_{\text{ff}}=4096$	427.93 M	frozen [†]
Flow I/O heads	Linear (action in/out, time MLP)	$32 \rightarrow 1024, 1024 \rightarrow 32$	2.17 M	frozen [†]
<i>Subtotal</i>			3.35 B	
<i>(c) RL token: prefix $[M, 2048] \rightarrow h_{r1} \in \mathbb{R}^{2048}$</i>				
Readout encoder	$2 \times (\text{self-attn} + \text{cross-attn} +$ GeGLU MLP)	$d=2048, h=8, \text{MLP} \times 4$	406.96 M	frozen
AR decoder	$2 \times (\text{causal self-attn} +$ cross-attn + GeGLU)	$d=2048, h=8, \text{MLP} \times 4$	415.35 M	train-only
<i>Subtotal</i>			822.32 M	
<i>(d) Residual RL policy: chunk-level TD3 on latent chunks</i>				
ChunkActor	3 proj. + 2-layer MLP (Dense-LN-swish)	$d_h=256$, zero-init tanh head	1.21 M	trainable
TwinCritic	2 independent Q networks, same trunk	$d_h=256$, 1.01 M each	2.02 M	trainable
<i>Subtotal[‡]</i>			3.23 M	
Total (all modules)			4.18 B	
Deployed at inference: VLA + hand decoders + RL-token encoder + actor			3.76 B	
Trainable during online RL			3.23 M	0.08%

[†] The full 3.35 B VLA is fine-tuned end-to-end during latent SFT and each DAgger round (30 k updates per phase, batch 64, cosine 5×10^{-5}) and kept *frozen* during RL-token training and online RL.

[‡] Target copies add another 3.23 M during training.

Table 4. Architectures used in the hand-representation comparison. Shapes and parameter counts are reported for one hand with $H = 32$ and a nine-dimensional latent per frame. In the chunk-based mappings, the additional 20-dimensional input is the measured hand state q_0 at the start of the chunk.

Method	Shape	Encoder / decoder	Parameters (enc. / dec. / total)	Embedding
Frame PCA	$20 \rightarrow 9 \rightarrow 20$	Nine principal components	—	None
Frame MLP	$20 \rightarrow 9 \rightarrow 20$	MLP widths (128, 64) / (64, 128)	11.9 K / 11.9 K / 23.8 K	None
Frame Transformer	$20 \rightarrow 9 \rightarrow 20$	2 layers per side, width 64, 4 heads, MLP $\times 2$	68.9 K / 68.3 K / 137.2 K	Finger positions; decoder queries
Chunk MLP	$(H \times 20, 20) \rightarrow H \times 9 \rightarrow H \times 20$	Flattened chunk; 2 hidden layers per side, width 512	0.90 M / 0.75 M / 1.65 M	None
Chunk Transformer	$(H \times 20, 20) \rightarrow H \times 9 \rightarrow H \times 20$	3 layers per side, width 128, 4 heads, MLP $\times 2$	426.3 K / 425.1 K / 851.4 K	Temporal positions; q_0 token

Table 5. Full training schedule for the reference policy. Each SFT policy and each DAgger round is trained for 30 k updates from the same public $\pi_{0.5}$ checkpoint.

Setting	Value	Setting	Value
<i>Initialisation and objective</i>			
Initialisation	same public $\pi_{0.5}$ checkpoint for every SFT policy and DAgger round	Updated parameters	all 3.35 B (vision encoder, backbone, action expert, flow I/O heads)
Objective	equation 10, conditional flow matching plus decoded hand channel	Loss weights	$\lambda_{\text{arm}}=1.0$, $\lambda_{\text{lat}}=0.25$, $\lambda_{\text{dec}}=0.75$
Flow time t_{flow}	0.999 Beta(1.5, 1) + 0.001		
<i>Optimisation</i>			
Steps	30 k per SFT policy or DAgger round	Batch size	64 chunks
Optimiser	AdamW, $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$	Weight decay	10^{-10}
Gradient clipping	global norm 1.0	Warmup	linear, 1000 steps
Decay	cosine over 30 k steps, peak 5×10^{-5} , floor 5×10^{-6}	Precision	bfloat16 parameters and activations, float32 optimiser state
EMA	decay 0.99	Hardware	8 $\times$ H20
Wall-clock	12 h per run		
<i>Data and inference</i>			
Views	3 cameras at 224^2	Prompt/state token IDs	$T_{\text{prompt}}^{\text{max}}=256$ (task, template, and quantized state; padded or truncated)
Chunk horizon H	32	Executed frames C	25
Sampling	SFT: uniform over episode start frames; DAgger: 50% new corrections, 50% prior data	Normalisation	per dimension, statistics from the training split only
Image augmentation	non-wrist views: 95% random crop and $\pm 5^\circ$ rotation; all views: color jitter (0.3/0.4/0.5 brightness/contrast/saturation)	Integration	10 Euler steps of the learned field
Decoding	hand latents through the frozen D^{hand} , arm blocks by identity		

Table 6. Default training configuration for offline RL-token pretraining. Optimiser, learning-rate, and EMA settings follow the base-policy defaults.

Setting	Value
<i>Optimisation</i>	
Optimiser	AdamW
AdamW parameters	$\beta_1=0.9, \beta_2=0.95, \epsilon=10^{-8}$
Weight decay	10^{-10}
Gradient clipping	global norm 1.0
EMA decay	0.99
Network dropout	0
<i>Learning-rate schedule</i>	
Warmup	linear, 1000 updates
Peak learning rate	5×10^{-5}
Cosine decay endpoint	30 k updates
Final learning rate	5×10^{-6} , held from 30 k to 40 k updates
<i>Training setup</i>	
Training budget	40 k updates
Global batch size	64 observations
Training seed	0
Parallelism	fully sharded data parallelism across eight devices

B.3 Offline RL-token pretraining

The RL-token readout is pretrained on observations from the reference-policy demonstration set using the reconstruction objective in equation 13. Observation preprocessing is the same as for the reference policy. The VLA remains frozen; only the readout token, encoder, output projection, and auxiliary decoder are optimised. Neither action labels nor rewards enter this objective.

Table 3 specifies the encoder and decoder architectures. Within each encoder block, the second attention sublayer takes its keys and values from the input to that block. The decoder is trained with teacher forcing, using right-shifted target features preceded by a learned beginning-of-sequence embedding and the RL token as cross-attention memory. Padded image-token positions are excluded from the encoder’s attention keys and the reconstruction loss. The minibatch loss is averaged over

all valid image-token positions and feature dimensions.

Table 6 summarises the default offline training configuration, including the optimiser, learning-rate schedule, and exponential moving average (EMA) settings inherited from the base policy. The learning rate follows linear warmup and cosine decay, then remains at its minimum for the final 10 k updates. Training uses a fixed update budget, without validation-based checkpoint selection or early stopping. Checkpoints are saved every 5000 updates and at the end of the 40 k-update schedule. The default configuration uses EMA weights for inference. After pretraining, the auxiliary decoder is discarded and the readout is frozen for online RL.

B.4 Online residual reinforcement learning

Table 7 gives the online training and collection settings after the reference policy and RL-token readout have been frozen.

Matched residual exploration. For the raw-space comparison of Section 4.4, we keep the TD3 updates, safety limits, and temporal structure of the exploration noise unchanged. Both residual methods use white noise on the arm increments, accumulated across the chunk, and the same direct per-dimension normalized Brownian noise ($\nu=2, 1/f^2$) on the hand block. The hand block is in latent coordinates for our method and in joint coordinates for the raw residual. Their noise scales are matched by the magnitude of the resulting perturbations in joint space after decoding, so the comparison changes the action representation without changing the physical exploration scale.

Table 7. Training and collection schedule for latent residual RL. The same optimisation settings are used for Transfer Marker between Hands and Rotate Marker in Hand; only the task-specific residual mask differs.

Setting	Value
<i>Objective and updates</i>	
Algorithm	TD3
Actor objective	Equation 20
Loss weights	$\beta_q=0.5, \beta_{\Delta z}=3.0, \beta_s=0.5$
Actor learning rate	1×10^{-4}
Critic learning rate	5×10^{-5}
Optimisers	Adam for actor and critics; no weight decay
Batch size	128 transitions
Discount	per-frame $\gamma=0.99$; chunk discount 0.99^{25}
Target update	Polyak coefficient $\tau_{\text{Polyak}}=5 \times 10^{-3}$
Actor update frequency	one actor update per critic update during joint training
Update cadence	200 gradient steps after each newly collected episode
Replay capacity	200 k transitions
Replay sampling	recency weighted 6:5:4:3:2 over the five latest episodes; latest episode guaranteed in each batch
<i>Action and interaction</i>	
Reference policy	frozen after SFT/Dagger
Chunk horizon	$H=32$
Executed frames	$C=25$ at 30 Hz
Residual scope	task-specific mask, Equation 17
Actor initialisation	zero output layer
Reference dropout	probability 0.5 per training sample
Residual bounds	arm increment 0.002 per frame; hand residual 0.10 in latent coordinates
Execution rate limit	maximum joint-position change $\pi/18$ rad per control frame
Warm-up	critics only after 500 replay transitions; actor enabled after manual stability check
Online budget	3000 transitions per task
Reward	terminal failure -1 ; success and non-terminal chunks 0
<i>Exploration and evaluation</i>	
Arm exploration	white noise on increments, accumulated across the chunk
Hand exploration	direct per-dimension Brownian noise ($\nu=2, 1/f^2$)
Noise scales	$\sigma^{\text{arm}}=0.01, \sigma^{\text{hand}}=0.05$
Evaluation noise	none
Evaluation frequency	every 1000 transitions
Evaluation trials	20 per checkpoint